

WAD Chat

LLM Chat Application

Assignment	Web Application Development
Student ID	ISU520196_3
Date	May 2026
Repository	github.com/brosjsy/llm_assignment_ISU520196_3

1. Project Overview

WAD Chat is a ChatGPT-style conversational application that runs a local Large Language Model (LLM) entirely on CPU — no cloud API or GPU required. Users register or sign in via password or GitHub OAuth, create named conversation threads, and exchange messages with the AI in real time through a streaming Server-Sent Events (SSE) interface.

Technology Stack

Layer	Technology	Purpose
Web framework	FastAPI 0.115	Async REST API + HTML routes
Database	PostgreSQL 16	Persistent storage for users, chats, messages
Migrations	Alembic 1.14	Versioned schema changes
Session store	Redis 7	Refresh token sessions (30-day TTL) + message cache
Auth tokens	JWT (python-jose)	Access token 30 min / Refresh token 30 days
Social auth	GitHub OAuth 2.0	Sign-in alongside password login
Password hash	Argon2 (passlib)	Secure credential storage
LLM runtime	llama-cpp-python	Local GGUF model, CPU inference
Templating	Jinja2	Server-rendered HTML views
HTTP client	httpx	Async GitHub API calls in AuthService

Architecture Pattern

The UI is **server-rendered HTML** (Jinja2 templates), so the project follows **MVC — Model · View · Controller**:

- **Models** — SQLAlchemy ORM classes (*User*, *Chat*, *Message*) in *app/users/models.py* and *app/chats/models.py*.
- **Views** — Jinja2 HTML templates in *templates/* (login, chat list, chat detail).
- **Controllers** — FastAPI route handlers in each *router.py*. Every endpoint instantiates the relevant service class and calls **exactly one** method. No business logic lives in the router.

JWT + Redis Session Flow

- On **register / login / GitHub OAuth**: AuthService issues an access token (30 min) and a refresh token (30 days). A random *session_id* UUID is stored in Redis with a 30-day TTL (key: *refresh:{session_id}*).
- Protected routes read the JWT from the *Authorization: Bearer* header or the *httponly access_token* cookie.
- On **refresh**: the old Redis key is deleted and a new session is issued (token rotation — limits replay attack windows).

- On **logout**: the Redis key is deleted immediately, invalidating the refresh token server-side even before it expires.

Bonus Features

- **Streaming output** — the LLM streams tokens via SSE; the browser renders them live with a blinking cursor.
- **Redis cache** — message history is cached per chat for 5 minutes, reducing DB load. Cache is invalidated on new message or chat deletion.
- **Auto-title** — the first user message (up to 40 chars) becomes the chat title.

2. Code Structure

The project is organised into feature packages under *app/*. Every feature package contains only files relevant to that feature; cross-cutting concerns (config, database, Redis) live directly under *app/*.

```
app/  
auth/  
depends.py FastAPI dependency – extract current user from JWT  
router.py Auth controllers (thin – one svc call per endpoint)  
schemas.py Pydantic I/O models  
security.py Argon2 hashing, JWT creation/decoding  
service.py All auth logic (register, login, refresh, logout, GitHub OAuth)  
chats/  
models.py SQLAlchemy ORM: Chat, Message  
router.py Chat controllers (thin – one svc call per endpoint)  
schemas.py Pydantic I/O models  
service.py Chat CRUD + LLM orchestration (ask, ask_stream)  
llm/  
service.py LLMService: llama-cpp singleton, build_prompt, generate  
users/  
models.py SQLAlchemy ORM: User  
config.py Pydantic BaseSettings – reads .env  
database.py Async engine, session factory, DeclarativeBase  
main.py FastAPI app, static files, router registration  
redis_client.py Module-level async Redis client  
templates/  
base.html HTML shell (charset, viewport, stylesheet)  
auth/login.html Login + Register tabs; GitHub OAuth button  
chats/list.html Chat list sidebar + welcome screen  
chats/detail.html Message thread; SSE streaming via EventSource  
static/css/style.css Dark-theme UI (CSS custom properties)  
alembic/ Database migration scripts  
versions/0001_initial.py Creates users, chats, messages tables  
docker-compose.yml PostgreSQL 16 + Redis 7 services  
requirements.txt Python dependencies (pinned versions)  
.env.example Environment variable template
```

Key Module Descriptions

app/auth/service.py — AuthService

Single class that owns all authentication business logic. Public methods: `register()`, `login()`, `refresh()`, `logout()`, `github_oauth_callback()`, `get_or_create_github_user()`. Internal: `_issue_tokens()` generates the JWT pair and stores the session in Redis.

app/chats/service.py — ChatService

Owns all chat and LLM orchestration logic. `ask()` is the full non-streaming turn: validate ownership → auto-title → save user message → get cached history → generate LLM answer → save assistant message → return `MessageResponse`. `ask_stream()` is an async generator that performs the same steps but yields SSE-formatted token strings, then persists the full answer on completion.

app/llm/service.py — LLMService

Thin wrapper around llama-cpp-python. Uses a module-level singleton so the model is loaded once. `generate()` returns a full completion; `generate_streaming()` yields tokens; `build_prompt()` formats the last 6 messages into a plain-text User/Assistant dialogue prompt.

app/auth/depends.py — get_current_user()

FastAPI dependency injected into every protected route. Reads the JWT from the Authorization: Bearer header first, then falls back to the `access_token` cookie. Returns the User ORM object or raises HTTP 401.

3. API Structure

The API is built with FastAPI and follows REST conventions. All protected endpoints accept the JWT either as an *Authorization: Bearer <token>* header or via the *httponly access_token* cookie set at login. Interactive documentation is available at **/docs** (Swagger UI).

Authentication Endpoints (prefix: /auth)

Method	Path	Auth	Description
POST	/auth/register	—	Register with login + password. Returns JWT pair. 400 if login
POST	/auth/login	—	Login. Returns JWT pair in response body + httponly cookies.
POST	/auth/refresh	—	Rotate refresh token. Body: {refresh_token}. Returns new token
POST	/auth/logout	JWT	Invalidate session in Redis; clear cookies.
GET	/auth/github	—	Redirect browser to GitHub authorization page.
GET	/auth/github/callback	—	Handle GitHub code exchange. Creates/retrieves user; redirect

Chat Endpoints (all require JWT)

Method	Path	Auth	Description
GET	/api/chats	JWT	List all chats for the authenticated user. Returns [{id, title, creat
POST	/api/chats	JWT	Create a new chat. Body: {title}. Returns 201 {id, title, created_
GET	/api/chats/{id}	JWT	Get chat with full message list. Returns {id, title, messages:[...]
DELETE	/api/chats/{id}	JWT	Delete chat and all messages. Clears Redis cache. Returns 20
POST	/api/chats/{id}/ask	JWT	Non-streaming LLM turn. Body: {content}. Returns assistant M
GET	/api/chats/{id}/stream	JWT	SSE stream. Query: ?content=... Yields data: <token> per toke

HTML Views (cookie JWT, server-rendered)

Method	Path	Auth	Description
GET	/	—	Login/Register page. Redirects to /chats if already authenticated
GET	/chats	JWT	Chat list with sidebar. Shows all user chats; welcome message
GET	/chats/{id}	JWT	Chat conversation. Renders messages; SSE streaming input a

Request / Response Schemas

RegisterRequest: { login: str, password: str, email: str | null }

LoginRequest: { login: str, password: str }

RefreshRequest: { refresh_token: str }

TokenResponse: { access_token: str, token_type: 'bearer' }

ChatCreate: { title: str = 'New Chat' }

ChatResponse: { id: int, title: str, created_at: datetime }

ChatDetail: ChatResponse + { messages: [MessageResponse] }

MessageResponse: { id: int, role: str, content: str, created_at: datetime }

AskRequest: { content: str }

4. Database Schema

PostgreSQL is used as the primary relational database. Schema changes are applied exclusively through **Alembic migrations** (the initial migration `0001_initial.py` creates all three tables). SQLAlchemy async ORM (asyncpg driver) is used for all queries.

Table: users

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing surrogate key
login	VARCHAR(150)	UNIQUE NOT NULL	Username; must be unique across the system
email	VARCHAR(255)	NULL	Optional email address
hashed_password	VARCHAR(255)	NULL	Argon2 hash; NULL for GitHub-only accounts
github_id	VARCHAR(100)	UNIQUE NULL	GitHub user ID; NULL for password accounts
is_active	BOOLEAN	NOT NULL	Soft-disable flag; default TRUE
created_at	TIMESTAMP	NOT NULL	Account creation time (UTC)

Table: chats

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing surrogate key
title	VARCHAR(255)	NOT NULL	Chat title; auto-set from first message
user_id	INTEGER	FK users.id CASCADE	Owner; deleting user cascades to chats
created_at	TIMESTAMP	NOT NULL	Thread creation time (UTC)

Table: messages

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing surrogate key
chat_id	INTEGER	FK chats.id CASCADE	Parent chat; deleting chat cascades to messages
role	VARCHAR(20)	NOT NULL	'user' or 'assistant'
content	TEXT	NOT NULL	Full message text
created_at	TIMESTAMP	NOT NULL	Message timestamp (UTC)

Entity-Relationship Summary

User **1** — **N** Chat (a user owns many chats; ON DELETE CASCADE)

Chat **1** — **N** Message (a chat has many messages; ON DELETE CASCADE)

Redis Data (non-relational)

Two key namespaces are used in Redis:

Key pattern	Value	TTL	Purpose
refresh:{session_id}	user_id (str)	30 days	Refresh token session – deleted on logout or
chat_messages:{chat_id}	JSON array	5 minutes	Cached message list – invalidated on new me

5. User Interface

The UI is server-rendered HTML with Jinja2 templates, styled with a dark theme using CSS custom properties. Minimal vanilla JavaScript handles form submission via the Fetch API and real-time streaming via the EventSource SSE API — no frontend framework is required.

Login / Register Page (GET /)

- Dark-themed card centred on screen.
- Two tabs: Login (default) and Register.
- Login tab: Username field, Password field, Sign in button, divider, Continue with GitHub button (GitHub OAuth).
- Register tab: Username, optional Email, Password, Create account button.
- On success: browser redirects to /chats (cookies set server-side).
- Error messages appear inline below the form in red.
- Enter key triggers the active form.

Chat List — Welcome Screen (GET /chats)

- Two-panel layout: 260 px sidebar on the left, main content area on the right.
- Sidebar header: app logo, + New Chat button.
- Sidebar body: scrollable list of all user chats with title and creation date.
- Sidebar footer: logged-in username, Logout button.
- Main area (no chat selected): welcome heading, explanatory text, Start a new chat button.

Chat Conversation (GET /chats/{id})

- Same sidebar layout; active chat is highlighted.
- Delete button (x) appears on hover over each chat item in the sidebar.
- Main panel — chat header: current chat title.
- Message thread: user messages right-aligned in purple bubbles; assistant messages left-aligned in dark bubbles; white-space preserved.
- Input area: auto-resizing textarea (max 160 px), Send button. Enter sends; Shift+Enter adds a newline.
- Streaming: tokens arrive via EventSource SSE and are appended character-by-character with a blinking cursor (textContent update, no HTML injection).

Swagger UI (GET /docs)

- FastAPI auto-generates interactive API documentation.
- All endpoints are listed and grouped by tag (auth, chats).
- Each endpoint shows method, path, request schema, response schema.
- The "Try it out" button allows sending real requests from the browser.

Note: actual screenshots of the running application can be viewed by starting the server locally (see README.md — Quick Start) and opening <http://localhost:8000>.

6. Security Design

Password storage

Passwords are hashed with Argon2 (memory-hard, resistant to GPU cracking) via *passlib[argon2]*. Plain-text passwords are never stored, logged, or transmitted after the initial request.

JWT tokens

HS256-signed JWTs. Access tokens expire in 30 minutes; refresh tokens expire in 30 days. Both are stored in httponly, SameSite=Lax cookies — inaccessible to JavaScript, preventing XSS-based token theft.

Session invalidation

Refresh token session IDs are stored in Redis with a matching TTL. Logout deletes the Redis key immediately, revoking the refresh token server-side regardless of its JWT expiry. Token rotation on refresh (old session deleted, new issued) limits the replay window.

GitHub OAuth

Authorization Code flow only. The GitHub client secret stays server-side. The received GitHub access token is used only to fetch the public user profile and is never stored in the database.

Input validation

All API request bodies are validated by Pydantic v2 before reaching service code. SQLAlchemy ORM with parameterized queries prevents SQL injection.